Lecture 7: Matrix Multiplication and Dynamic programming

Instructor: Sourav Chakraborty, Sanjukta Roy *Scribe: Gururaj, Anshul*

1 Matrix Multiplication

1.1 Classical Matrix Multiplication

Let A and B be two square matrices of size $n \times n$. The product matrix $C = AB$ is defined as:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Since the product of two $n \times n$ matrices has n^2 entries, any algorithm for matrix multiplication must compute all of them. Therefore, matrix multiplication has a lower bound of $\Omega(n^2)$ time.

1.1.1 Algorithm Description

The classical algorithm uses three nested loops.

1.1.2 Pseudocode

Algorithm 1 Classical Matrix Multiplication

Input: Matrices $A, B \in \mathbb{R}^{n \times n}$
Output: Matrix $C = AB$

```
for i = 1 to n do
    for j = 1 to n do
        C_ij ← 0
        for k = 1 to n do
            C_ij ← C_ij + A_ik × B_kj
        end for
    end for
end for
return C
```

1.1.3 Time Complexity Analysis

- There are n^2 elements in the result matrix C.
- Each element requires n multiplications and additions.

Thus, the total number of operations is:

$$n \times n \times n = n^3$$

Therefore, total time complexity:

$$\boxed{(O(n^3))}$$

1.1.4 Space Complexity

The algorithm stores the input matrices A and B and the result matrix C, each of size $n \times n$. Hence, the total auxiliary space required is proportional to n^2.

$$\boxed{O(n^2)}$$

1.2 Strassen's Matrix Multiplication

Strassen's algorithm reduces the number of multiplications using a divide-and-conquer strategy.

1.2.1 Divide and Conquer Approach

Each matrix is divided into four submatrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

1.2.2 Pseudocode

Algorithm 2 Strassen's Matrix Multiplication

Input: Matrices $A, B \in \mathbb{R}^{n \times n}$
Output: Matrix $C = AB$

if $n = 1$ **then**
    return $C = A \times B$
else
    Divide A into $A_{11}, A_{12}, A_{21}, A_{22}$
    Divide B into $B_{11}, B_{12}, B_{21}, B_{22}$
    $P_1 \leftarrow (A_{11} + A_{22})(B_{11} + B_{22})$
    $P_2 \leftarrow (A_{21} + A_{22})B_{11}$
    $P_3 \leftarrow A_{11}(B_{12} - B_{22})$
    $P_4 \leftarrow A_{22}(B_{21} - B_{11})$
    $P_5 \leftarrow (A_{11} + A_{12})B_{22}$
    $P_6 \leftarrow (A_{21} - A_{11})(B_{11} + B_{12})$
    $P_7 \leftarrow (A_{12} - A_{22})(B_{21} + B_{22})$
    $C_{11} \leftarrow P_1 + P_4 - P_5 + P_7$
    $C_{12} \leftarrow P_3 + P_5$
    $C_{21} \leftarrow P_2 + P_4$
    $C_{22} \leftarrow P_1 - P_2 + P_3 + P_6$
    Combine $C_{11}, C_{12}, C_{21}, C_{22}$ into C
    return C
end if

1.2.3 Time Complexity Analysis

The recurrence relation is:

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

Using the Master Theorem:

$$\boxed{(O(n^{\log_2 7}) \approx O(n^{2.81}))}$$

1.2.4 Space Complexity

$$\boxed{(O(n^2))}$$

There are some faster matrix multiplication algorithms which can compute multiplication as fast as $O(n^{2.37286})$ time.

2 Longest Common Subsequence

Given two sequences

$$A = \langle a_1, a_2, \ldots, a_m \rangle \quad \text{and} \quad B = \langle b_1, b_2, \ldots, b_n \rangle$$

the **Longest Common Subsequence (LCS)** problem is to find the longest subsequence common to both sequences.

A subsequence is obtained by deleting zero or more elements from a sequence without changing the relative order of the remaining elements.

2.1 Example

Drop some elements such that the remaining subsequence is same.

$$A = \texttt{APPLE}$$

$$B = \texttt{PEOPLE}$$

Common subsequences:

$$\texttt{PPLE}, \quad \texttt{PPE}, \quad \texttt{PE}$$

Longest common subsequence:

$$\boxed{\texttt{PPLE}}$$

Length of LCS:

$$4$$

2.2 Approach 1: Naive Approach

The naive approach is based on the brute-force principle. All possible subsequences of both given strings are generated, and the longest subsequence that appears in **both** strings is selected as the Longest Common Subsequence (LCS).

Since a string of length n has 2^n subsequences, this approach checks all possible combinations, making it computationally expensive.

2.2.1 Algorithm

Algorithm 3 Naive Longest Common Subsequence

```
1: function NaiveLCS(A, B)
2:     Generate all subsequences of A and store in set S_A
3:     Generate all subsequences of B and store in set S_B
4:     maxLen ← 0
5:     for all s ∈ S_A do
6:         if s ∈ S_B and |s| > maxLen then
7:             maxLen ← |s|
8:         end if
9:     end for
10:    return maxLen
11: end function
```

2.2.2 Time Complexity

A string of length n has 2^n subsequences. In the worst case, all subsequences of both strings must be compared.

$$\boxed{O(2^m \times 2^n)}$$

2.2.3 Space Complexity

All subsequences of both strings must be generated and stored. A string of length m has 2^m subsequences and a string of length n has 2^n subsequences. Thus, storing these subsequences requires exponential space.

$$\boxed{O(2^m + 2^n)}$$

2.3 Approach 2: Recursive Method

Let

$$A = \langle a_1, a_2, \ldots, a_m \rangle \quad \text{and} \quad B = \langle b_1, b_2, \ldots, b_n \rangle$$

If the last elements of both sequences are equal, i.e.,

$$a_m = b_n,$$

then the Longest Common Subsequence must contain this element. Hence, the problem reduces to finding the LCS of the remaining prefixes:

$$LCS(A, B) = 1 + LCS(\langle a_1, \ldots, a_{m-1} \rangle,\ \langle b_1, \ldots, b_{n-1} \rangle)$$

If the last elements are not equal, i.e.,

$$a_m \neq b_n,$$

then the Longest Common Subsequence is obtained by considering either of the following cases:

$$LCS(\langle a_1, \ldots, a_{m-1} \rangle,\ \langle b_1, \ldots, b_n \rangle)$$

or

$$LCS(\langle a_1, \ldots, a_m \rangle,\ \langle b_1, \ldots, b_{n-1} \rangle)$$

The maximum of these two values gives the length of the Longest Common Subsequence:

$$LCS(A, B) = \max\Big(LCS(a_1, \ldots, a_{m-1};\ b_1, \ldots, b_n),\ LCS(a_1, \ldots, a_m;\ b_1, \ldots, b_{n-1})\Big)$$

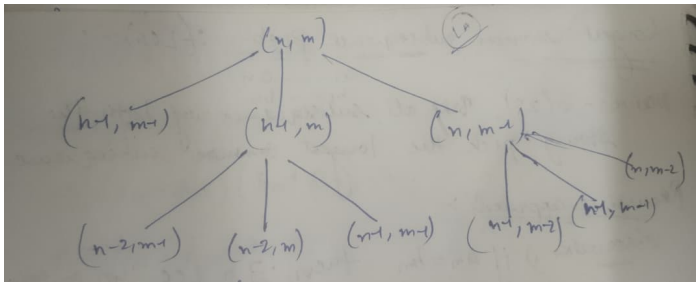

Figure 1: Recursion Tree

Algorithm 4 Recursive Longest Common Subsequence

```
1: function LCS(A, B, m, n)
2:     if m = 0 or n = 0 then
3:         return 0
4:     else if a_m = b_n then
5:         return 1 + LCS(A, B, m − 1, n − 1)
6:     else
7:         return max(LCS(A, B, m − 1, n), LCS(A, B, m, n − 1))
8:     end if
9: end function
```

2.3.1 Time Complexity Recurrence

In the worst case, when characters never match, the recurrence relation becomes:

$$T(m+n) = T((m-1)+n) + T(m+(n-1)) + O(1)$$

Let

$$k = m + n$$

Then,

$$T(k) = 2T(k-1) + O(1)$$

Solving this recurrence:

$$T(k) = O(2^k)$$

Hence, the time complexity of the naive recursive LCS algorithm is:

$$\boxed{O(2^{m+n})}$$

This recursion tree clearly shows that subproblems such as $(m-1, n-1)$ are recomputed multiple times.

2.3.2 Space Complexity Analysis

The space complexity is determined by the recursion stack. In the worst case, the maximum depth of recursion is $\min(m, n)$.

$$\boxed{\text{Space Complexity} = O\left(\min(m, n)\right)}$$

2.4 Approach 3: Dynamic Programming (Top-Down / Memoization)

2.4.1 Algorithm

Algorithm 5 LCS using Memoization

```
1: function LCS_Memo(A, B, m, n, dp)
2:     if m = 0 or n = 0 then
3:         return 0
4:     end if
5:     if dp[m][n] ≠ −1 then
6:         return dp[m][n]
7:     end if
8:     if A[m] = B[n] then
9:         dp[m][n] ← 1+ LCS_Memo(A, B, m − 1, n − 1, dp)
10:    else
11:        dp[m][n] ← max(LCS_Memo(A, B, m − 1, n, dp), LCS_Memo(A, B, m, n −
       1, dp))
12:    end if
13:    return dp[m][n]
14: end function
```

The function LCS_Memo computes the length of the Longest Common Subsequence (LCS) between two sequences A and B of lengths m and n using **top-down dynamic**

programming with memoization.

- If either sequence has length zero ($m = 0$ or $n = 0$), then no common subsequence exists, and the function returns 0.
- The table $dp[m][n]$ stores the LCS length of the prefixes $A[1 \dots m]$ and $B[1 \dots n]$. If this value has already been computed ($dp[m][n] \neq -1$), it is returned immediately, avoiding redundant recursive calls.
- If the last characters of the prefixes match ($A[m] = B[n]$), then the LCS must include this character. Hence,

$$dp[m][n] = 1 + \text{LCS\_Memo}(A, B, m-1, n-1, dp).$$

- If the last characters do not match ($A[m] \neq B[n]$), then the LCS is the maximum of:

$$\text{LCS\_Memo}(A, B, m-1, n, dp) \quad \text{and} \quad \text{LCS\_Memo}(A, B, m, n-1, dp).$$

- The computed value is stored in $dp[m][n]$ and returned.

2.4.2 Time Complexity Analysis

Each subproblem is uniquely identified by the pair (m, n). There are only:

$$(m+1)(n+1) = O(mn)$$

distinct subproblems.

$$\boxed{\text{Time Complexity} = O(mn)}$$

2.4.3 Space Complexity Analysis

The memoization table stores mn entries, and the recursion stack requires at most $m + n$ space.

$$\boxed{\text{Space Complexity} = O(mn)}$$

2.5 Approach 4: Dynamic Programming (Bottom-Up / Tabulation)

This algorithm computes the length of the **Longest Common Subsequence (LCS)** between two sequences A and B of lengths m and n using **bottom-up dynamic programming (tabulation)**.

- A two-dimensional table $dp[i][j]$ is constructed, where $dp[i][j]$ denotes the length of the LCS of the prefixes $A[1 \dots i]$ and $B[1 \dots j]$.
- The table is initialized with zeros in the first row and first column. This represents the base case that if either sequence is empty ($i = 0$ or $j = 0$), the LCS length is 0.
- The table is filled row by row. If the current characters match ($A[i] = B[j]$), then the LCS is extended by one:

$$dp[i][j] = 1 + dp[i-1][j-1].$$

- If the characters do not match ($A[i] \neq B[j]$), then the LCS length is the maximum of the two possible subproblems:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1]).$$

- After filling the table, the value stored in $dp[m][n]$ gives the length of the Longest Common Subsequence of A and B.

2.5.1 Algorithm

Algorithm 6 LCS using Tabulation

```
for i = 0 to m do
    for j = 0 to n do
        if i = 0 or j = 0 then
            dp[i][j] ← 0
        else if A[i] = B[j] then
            dp[i][j] ← 1 + dp[i − 1][j − 1]
        else
            dp[i][j] ← max(dp[i − 1][j], dp[i][j − 1])
        end if
    end for
end for
return dp[m][n]
```

DP Table Visualization The table represents the dynamic programming matrix used for solving the LCS problem. Each cell (i, j) stores the length of the longest common subsequence for the prefixes $A[1 \dots i]$ and $B[1 \dots j]$. To compute (i, j), we use previously computed values $(i-1, j)$, $(i, j-1)$, and $(i-1, j-1)$. Thus, the table is filled systematically from smaller subproblems to larger ones.

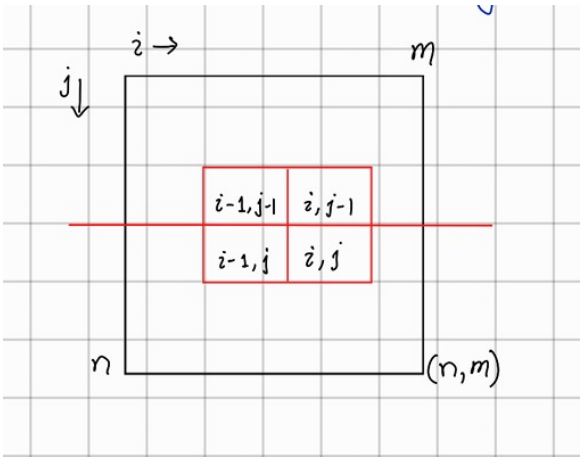

Figure 2: LCS Dynamic Programming Table

2.5.2 Time Complexity Analysis

$$\boxed{\text{Time Complexity} = O(mn)}$$

2.5.3 Space Complexity Analysis

$$\boxed{\text{Space Complexity} = O(mn)}$$

3 Longest Increasing Subsequence

Given a sequence

$$A = \langle a_1, a_2, \ldots, a_n \rangle$$

find the longest increasing subsequence

3.1 Method 1

Sort the array to obtain

$$B = \{b_1, b_2, \ldots, b_n\}$$

Then apply the **Longest Common Subsequence (LCS)** algorithm between the original array A and the sorted array B.

$$\text{LIS}(A) = \text{LCS}(A, B, n, n)$$

3.2 Method 2

Solve the problem using **Dynamic Programming (DP)**.

Let

$$dp[i] = \text{length of the longest increasing subsequence ending at } a_i$$

Base case:

$$dp[i] = 1 \quad \text{for all } 1 \leq i \leq n$$

Recurrence relation:

$$dp[i] = 1 + \max\{dp[j] \mid j < i \text{ and } a_j < a_i\}$$

$$\text{Length of LIS} = \max_{1 \le i \le n} dp[i]$$

4 Knapsack Problem

Let the items be

$$I_1 = (w_1, c_1), \quad I_2 = (w_2, c_2), \quad \ldots, \quad I_n = (w_n, c_n)$$

where w_i is the weight and c_i is the profit of item i.

Given a knapsack of capacity M kg, choose a subset

$$S \subseteq \{1, 2, \ldots, n\}$$

to

$$\max \sum_{i \in S} c_i$$

subject to

$$\sum_{i \in S} w_i \le M, \qquad w_i \in \mathbb{N}$$